The OWASP Testing Guide (V.4) provides four security testing techniques: manual inspections and reviews, threat modeling, code review and penetration testing. (OWASP Web Security Testing Guide, pg. 13-23, | OWASP Foundation. n.d.). Once we discuss the types of tests, we will discuss how they can be used together.

Manual inspections and reviews are the first security test. This test focuses on the human side of the development process. These tests typically look at the security of people, policies and processes. Manual inspections can also include architectural designs. While the concept is simple, these tests can be extremely effective. By asking the developer how something works and why it was implemented in a certain way, a tester can quickly determine if security concerns are likely to be found. However, not everything the tester is shown or told will be accurate. These tests rely on a trust-but-verify approach. Manual inspections and reviews don't require any technology, they can be applied to many situations, they are flexible and promote teamwork. However, they are also time consuming, and they require significant human thought to be effective.

Threat modeling is the process of creating a threat model, or a model that shows all the possible threats to a system or application. The process starts by deconstructing the application. Using manual inspections, breakdown the application into its assets, functionality and connections. Once the application is broken down sort the assets into groups of tangible and intangible assets and rank them based on their importance to the business. Explore the potential vulnerabilities and threats to the application. Using these vulnerabilities and threats you can then create mitigation strategies. Threat models help the developers focus their attention on the largest potential threats to the application very early into development. Allowing them to use their limited resources effectively.

Source code reviews are very important. Many serious vulnerabilities cannot be detected in any other way. Source code reviews look directly at the code of the application. Most security experts agree that there is no substitute for looking at the code. Many security problems are difficult to discover with the other forms of testing. With the source code a tester can easily see what is happening (or what is supposed to happen) without having to guess. Examples of issues that code reviews can find include: “concurrency problems, flawed business logic, access control problems, and cryptographic weaknesses as well as backdoors, Trojans, Easter eggs, time bombs, logic bombs, and other forms of malicious code” (OWASP Web Security Testing Guide | OWASP Foundation, n.d.-b). There are a few downsides of source code reviews. They require high skill developers, they can miss issues in libraries, and the source code might change between the review and the deployment.

Penetration testing is testing an application remotely to find security vulnerabilities, without knowing how the application works internally. These tests have the tester act like attackers, trying to find and exploit vulnerabilities. Penetration testing is also known as black box testing or ethical hacking. However, penetration testing is more effective in the network security space. Pen-testing a network is mostly finding and exploiting known vulnerabilities in technologies. But web applications are almost always custom. Pen-testing a web application is more like research. Penetration testing should not be the end-all-be-all of security testing. Penetration testing is best used to target known vulnerabilities in the application to see if they have been fixed.

By themselves these testing methods are good but together they can be great. Each of these tests can be used together to cover the entire development timeline. You start during the design phase with threat modeling. You make a threat model that the developers can use during

development to focus their resources on the highest threat possibilities. Then once the code is written, source code reviews can begin. These reviews can find problems that may have been overlooked during development. These problems can then be fixed before the full release. Shortly before the release penetration testing can be used to look for vulnerabilities from the outside of the application. During the entire development process manual inspections can be performed. By themselves each of the tests fail to cover everything. However, when used together they cover each other's weaknesses while providing their strengths.

STRIDE is a threat model that looks at six major areas of vulnerability: spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privilege. STRIDE models are useful for identifying threats based on the attacker's goal. Using the list of vulnerabilities found in the previous phase of this project I was able to make a STRIDE analysis of Hackazon.

S – Spoofing identity

The admin login page is discoverable and interactable from the internet. Adding /admin to the end of the Hackazon URL takes you to the admin log in page. regardless of your current user type. This gives the attacker access to the admin hub if they can brute force their way into the system. This is a high-risk vulnerability. Some mitigation techniques are appropriate authentication and remove the admin log in access directly from the internet.

T – Tampering with data

Hackazon has many vulnerabilities of this type. The Hackazon search function accepts values from outside the bounds of what it can search for. This allows malicious users to place scripts into the search function. The function doesn't attempt to validate input. This can allow a

malicious user to manipulate the client sided nature of Hackazon. Mitigation is relatively simple. Server-side input validation, it should enforce the type, length and format of the search function.

Hackazon does not assume user input is malicious and will run scripts and SQL injections. The system does not sanitize the input before running it, allowing injection-based attacks to be performed on the system. These vulnerabilities affect the integrity of the application. Hackazon is directly targetable by SQL injection attacks. The log in page is susceptible to these attacks. Hackazon is also vulnerable to scripting attacks. Using <script>alert(1)</script> in the item search gives a pop up that shows 1. Mitigation techniques include proper authorization as well as server sided tamper resistant protocols.

Hackazon relies heavily on client-side validation. Hackazon is vulnerable to client sided page manipulation. Using dev tools, you can edit the price of items because they are stored on the webpage rather than the server. You can also manipulate the search parameters which are stored on the webpage. Hackazon does not use defense in depth. There are problems with input validation across the entire website.

R – Repudiation

NONE

I – Information Disclosure

Hackazon lacks any encryption for data in transit. The application uses HTTP instead of HTTPS. This means that credentials, session tokens or other sensitive data is open to interception. To secure communications Hackazon should enforce the use of HTTPS with an encryption algorithm. Hackazon also transmits data using the GET protocol. This opens sensitive

information to be exposed by URLs, logs or browser history. Hackazon doesn't canonicalize itself. This opens the site up to hackers skipping filters using alternate versions of the site.

D – Denial of Service

NONE

E – Elevation of Privilege

NONE

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-AUTR-001	Access Rights	STC-AUTR-001	Access Rights	Verify that the system does not allow access to system resources without checking the assigned rights and privileges of the authenticated user. In case of Web applications, brute force access to different URLs to which users don't have access to is an important verification test. In the case of Web applications, verify that no user-tamperable parameters/hidden parameters that control access control exist.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-AUTR-008	User and Group Privileges	STC-AUTR-008	User and group privileges	Verify that the system has administrative features to assign user and group privileges (i.e., access permissions) to user IDs (not authentication information).

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-AUTR-009	Role-Based Access Control	STC-AUTR-009	Role-Based Access Control	Verify that the system provides an enforceable mechanism through which users can be segmented into roles (e.g., administrator) involving access to security features and other administrative functions.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-ATEN-001	Credential Security	STC-ATEN-001	Credential Security	Verify that the passwords for all user accounts are stored in the database using a one-way hash function. Verify that each user ID has a unique salt for better security.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-ATEN-006	Protection of Credentials	STC-ATEN-006	Protection of Credentials	Verify that the passwords for all user accounts are never stored or transmitted in clear text. This shall include but not be limited to verifying databases, logfile, error messages, administrative console message, flat files on the application file systems, etc.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-INTG-002	Source Identification	STC-INTG-002	Source Identification	Verify that the system has the capability to propagate, when requested, the original user identifier to the destination.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-INTG-007	Integrity Checks	STC-INTG-007	Integrity Checks	Verify that the system has the capability to protect data integrity by performing data integrity checks; reject the data if the integrity check fails.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and

				Comments
SR-IDEN-001	Unique User ID	STC-IDEN-001	Unique User ID	Verify that no two entities in the software ecosystem carry that same internal or external identification. This would involve verification of aliases, if any, and how those are mapped to actual users.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-AUDT-001	Audit Log	STC-AUDT-001	Audit Log	Verify that the system maintains an audit log that provides adequate information for establishing audit trails on security breaches and user activity.

SR ID	SR Name	STC ID	STC Name	STC Description,

				Constraints, and Comments
SR-AUDT-002	Logging of Authentication Information	STC-AUDT-002	Logging of Authentication Information	Verify that the system maintains the confidentiality of authenticators (e.g., passwords) by excluding them from being recorded in the audit log.

SR ID	SR Name	STC ID	STC Name	STC Description, Constraints, and Comments
SR-AUDT-003	Logging of Specific Events	STC-AUDT-003	Logging of Specific Events	Verify that the system allows the administrator to configure the audit log to record specified events such as: ■All sessions established ■Failed user authentication attempts ■Unauthorized attempts to access resources (e.g. software, data, process) ■Administrator actions

				■Administrator disabling of audit logging ■Events generated (e.g., commands issued) to make changes in users' security profiles and attributes ■Events generated to make changes in the security profiles and attributes of system interfaces ■Events generated to make changes in permission levels needed to access a resource ■Events generated that make changes to the system security configuration ■Events generated that make modifications to the system software Verify that such events are logged by verifying the log location.

References

Initiative, J. T. F. T. (2018). Risk management framework for information systems and organizations: https://doi.org/10.6028/nist.sp.800-37r2

OWASP Web Security Testing Guide | OWASP Foundation. (n.d.). https://owasp.org/www-project-web-security-testing-guide/

Threat Modeling Process | OWASP Foundation. (n.d.). https://owasp.org/www-community/Threat_Modeling_Process#stride